CAPSTONE PROJECT REPORT

NavRL-Enhanced Urban UAV Navigation: A Hybrid Deep Reinforcement Learning and Path-Planning Framework for Safe Autonomous Flight in Structured Environments

Department of Electrical and Computer Engineering

Academic Year 2025–2026

Field	Details
Project Title	NavRL-Enhanced Urban UAV Navigation
Submission Date	April 2026
Simulator	Microsoft AirSim (Unreal Engine 4, City Environment)
Base Framework	NavRL [1] — Carnegie Mellon University (CMU)
Test Platform	AirSim City, 12-waypoint urban roam mission (~800 m)

Supervisor Signature: _____

Student Signature: _____

Abstract

This report presents the capstone evaluation and extension of **NavRL** [1], a deep reinforcement learning (DRL) navigation framework for unmanned aerial vehicles (UAVs) originally developed at Carnegie Mellon University. NavRL employs Proximal Policy Optimization (PPO) [17] with carefully designed state representations, LiDAR-based static obstacle perception, and a velocity-obstacle safety shield to achieve collision-free autonomous flight, validated through zero-shot sim-to-real transfer on a physical quadcopter [1].

The contribution of this capstone project is threefold: (1) a faithful AirSim simulation deployment of the NavRL model for quantitative benchmarking in a photorealistic urban environment; (2) the design and implementation of a hybrid city planner that integrates NavRL's reactive RL policy with A* global path planning, a city altitude controller, and a Pure Pursuit lookahead module; and (3) a systematic multi-suite evaluation comparing the hybrid system against the pure RL baseline across standard, domain-randomization, ablation, and sensor-noise conditions.

Results demonstrate that the hybrid system achieves a **75.00% goal-success rate** with **0.69 collisions/km** — a **2× improvement in success** and **13× reduction in collision rate** relative to the pure RL baseline (36.66%, 9.31 collisions/km) — validating the thesis that structured urban navigation requires global planning in combination with reactive RL control.

Table of Contents

1. Introduction

Autonomous unmanned aerial vehicles (UAVs) are increasingly deployed in applications ranging from urban delivery and infrastructure inspection to search-and-rescue operations [2][3]. Safe flight in structured environments — where buildings, elevated infrastructure, and dynamic elements coexist — demands navigation systems that are simultaneously reactive at the local level (obstacle avoidance within meters) and deliberate at the global level (route planning across hundreds of meters).

Traditional approaches decompose this problem into a pipeline of modular components: a global planner (e.g., A*, RRT*), a local planner (e.g., potential fields, ESDF gradient), and a low-level controller. While these systems are interpretable and tunable, they suffer from parameter brittleness and performance degradation when environmental assumptions are violated [1].

Deep reinforcement learning offers an alternative: through trial-and-error in simulation, a policy network can learn a mapping from raw sensory observations directly to velocity commands, implicitly encoding obstacle avoidance without hand-crafted rules. The NavRL framework [1], published in *IEEE Robotics and Automation Letters* (2025) by Xu et al. at CMU, achieves precisely this — demonstrating zero-shot sim-to-real transfer on a physical quadcopter. This capstone project builds directly on their published checkpoint and framework, deploying it in a photorealistic urban simulation environment and augmenting it with a structured city planner.

However, purely reactive RL has a fundamental limitation: its obstacle perception range is bounded by the LiDAR maximum range (4 m in NavRL [1]), which is insufficient to route around large, extended obstacles or to select globally efficient paths in a dense urban canyon. This work hypothesizes and empirically confirms that **a hybrid system combining NavRL's reactive policy with A* global**

planning achieves significantly superior performance in structured urban environments, without retraining the underlying policy.

1.1 Objectives

- Deploy and benchmark the NavRL checkpoint [1] in Microsoft AirSim's urban environment.
- Design a hybrid city planner wrapping NavRL with A* global path planning, altitude management, and Pure Pursuit lookahead.
- Evaluate both systems across standard, domain-randomization, ablation, and sensor-noise test suites.
- Analyze the contribution of each architectural component through systematic ablation.
- Outline a hardware platform for real-world deployment based on the validated system.

1.2 Significance

Pure RL navigation has been demonstrated in controlled, low-density environments [1][11]. This work is among the first to systematically quantify the performance gap between pure RL and a hybrid RL+planner architecture in a photorealistic urban environment with 12 distinct waypoints, multi-story buildings, and structured road topology. The 13× collision rate reduction and 2× success improvement provide a quantitative argument for the necessity of global planning in urban UAV navigation.

2. Literature Review / Related Work

2.1 Traditional UAV Navigation

Rule-based methods for UAV navigation in dynamic environments rely on hierarchical modules with handcrafted algorithms. Wang et al. [4] demonstrate vision-aided autonomous flight in dynamic environments with onboard vision. Xu et al. [5] address gradient-based B-spline trajectory optimization for dynamic obstacle avoidance (ViGO). These methods achieve good performance in their target settings but require careful parameter tuning and can fail under environmental distribution shifts.

The EGO-Planner [6] is a widely benchmarked ESDF-free gradient-based local planner for quadrotors that operates without explicit signed-distance-field computation. While efficient in open environments, its map-update mechanism becomes noisy in the presence of dynamic obstacles, causing it to stall — a failure mode quantified in NavRL's benchmark study [1] (Table 2, Section 3.6).

2.2 Deep Reinforcement Learning for UAV Navigation

Q-learning and value-learning approaches [7][8][9] have demonstrated UAV navigation but are constrained to discrete action spaces, limiting maneuverability. Policy gradient methods using actor-critic structures [10] support continuous action spaces. Kaufmann et al. [11] trained a policy in simulation that outperforms human pilots in drone racing, demonstrating the capability ceiling of RL-based drone control.

For collision avoidance, He et al. [12] introduce a reach-avoid network as a recovery policy. Kochdumper et al. [13] use reachability analysis for safe action projection, though with exponential scaling in action dimensions. Recovery-RL [11] integrates learned recovery zones into the RL framework for safe exploration.

2.3 Sim-to-Real Transfer

The sim-to-real gap is a central challenge for RL-based UAV systems. Camera-based methods [14][15][16] are particularly susceptible due to rendering differences. NavRL [1] addresses this by adopting ray-cast LiDAR representations — which have minimal simulation-to-reality discrepancy — rather than camera images. This design choice enables zero-shot transfer: the model trained entirely in NVIDIA Isaac Sim operates directly on a real quadcopter without fine-tuning.

2.4 Hybrid Architectures

Several works have explored combining RL with classical planning. Chen et al. [21] propose risk-aware trajectory sampling for quadrotor obstacle avoidance. The FUEL framework [1-ref1] integrates incremental frontier exploration with hierarchical planning. This capstone's hybrid architecture is most directly related to NavRL [1] but is unique in extending NavRL with A* path planning over a dynamically updated occupancy grid specifically designed for the urban AirSim environment.

2.5 Positioning of This Work

This capstone extends NavRL to structured urban environments where the 4 m reactive horizon is insufficient for global navigation. The hybrid architecture introduced here is orthogonal to the original training procedure — the RL policy is used as-is, augmented by a planning layer it was not trained to collaborate with. This tests the generalizability of the NavRL policy outside its training distribution and provides a quantifiable measure of what pure RL alone cannot achieve in a city-scale environment.

3. The NavRL Framework — Theory and Mathematical Foundations

This section presents the theoretical foundations of NavRL as published in [1] (Xu et al., IEEE RA-L 2025), included here to provide the mathematical basis of the system under evaluation. All equations and results in this section are attributed to [1] unless otherwise noted.

3.1 Problem Formulation

The navigation task is formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ), where S is the state space, A is the continuous action space, $P(s_{t+1} \mid s_t, a_t)$ is the transition model, $R(s_t, a_t)$ is the reward function, and $\gamma \in [0,1]$ is the discount factor. The optimal policy maximises the expected discounted cumulative reward [1]:

$$\pi^* = \text{argmax}_\pi \blacksquare[\Sigma_{t=0}^{T} \gamma^t R(s_t, a_t)] \quad (1)$$

3.2 State Representation

3.2.1 Internal State

The robot's internal state captures its goal-relative geometry and velocity. All vectors are expressed in the **goal coordinate frame** $(\cdot)^G$, where the X-axis aligns with the robot-to-goal direction. This transformation was shown by Xu et al. [1] to reduce dependency on absolute world coordinates and improve RL convergence speed:

$$S_{int} = [\ (P_g^G - P_r^G) / \blacksquare P_g^G - P_r^G \blacksquare, \blacksquare P_g^G - P_r^G \blacksquare, V_r^G\ ]^T \quad (2)$$

where P_r and P_g denote robot and goal positions respectively, and V_r is the robot velocity. The full internal state vector is **8-dimensional**: 3D unit vector toward goal, 2D distance to goal (horizontal + vertical), and 3D velocity in goal frame.

3.2.2 Dynamic Obstacle State

Dynamic obstacles are represented as a matrix $S_{dyn} \in \blacksquare^{N_d \times M}$, where each row corresponds to the i-th closest obstacle [1]:

$$D_i = [\ (P_{oi}{}^G - P_r{}^G)/\blacksquare \ldots \blacksquare,\ \blacksquare P_{oi}{}^G - P_r{}^G \blacksquare,\ V_{oi}{}^G,\ \dim(o_i)\ ]^T \quad (3)$$

If fewer than N_d obstacles are detected, remaining entries are zero-padded.

3.2.3 Static Obstacle State — LiDAR Ray Casting

Static obstacles are encoded via 3D ray casting from the robot's position against an occupancy voxel map. Rays are cast horizontally at 360° and at multiple vertical elevation angles θ_v. The resulting range matrix is [1]:

$$S_{stat} = [R_{\theta 0}, \ldots, R_{\theta Nv}],\ S_{stat} \in \blacksquare^{N_h \times N_v} \quad (4)$$

where $N_h = \blacksquare 360 \ / \ \Delta\theta_h \blacksquare$ and N_v is the number of vertical planes. The inverted representation fed to the network is:

$$S_{stat}{}^{inv}(i,j) = \max(R_{max} - S_{stat}(i,j),\ 0.1) \quad (5)$$

so that high values indicate **proximity** rather than distance, matching the sign convention expected by the trained policy.

Parameter	Value
$\Delta\theta_h$	10° (36 horizontal bins)
θ_v angles	–10°, 0°, 10°, 20° (4 vertical bins)
Max ray length	4.0 m
Bin 0 orientation	Aligned toward goal direction

Table 1: *NavRL LiDAR ray-casting parameters as deployed in [1].*

3.3 Action Representation

At each time step, the policy outputs a normalised velocity $V\blacksquare^G{}_{ctrl} \in [0,1]^3$ in the goal coordinate frame. The final velocity command is [1]:

$$V^G{}_{ctrl} = v_{lim} \cdot (2 \cdot V\blacksquare^G{}_{ctrl} - 1),\ v_{lim} = 2.0 \text{ m/s} \quad (6)$$

To constrain the output to [0,1], the actor network produces parameters (α, β) for a **Beta distribution** [1][16]. Using the Beta distribution rather than a Gaussian for

bounded action spaces has been shown to be bias-free and achieve faster convergence [16]:

$$V\blacksquare^{G}_{ctrl} \sim \text{Beta}(\alpha, \beta),\ \alpha, \beta > 0 \tag{7}$$

During deployment, the **mean** of the Beta distribution is used:

$$V\blacksquare^{G}_{ctrl} = \alpha / (\alpha + \beta) \tag{8}$$

The velocity must then be transformed from the goal frame to the world frame [1]:

$$V^{world}_{ctrl} = R^{-1}_{G \to W} \cdot V^{G}_{ctrl} \tag{9}$$

3.4 Reward Function

The total reward at each time step consists of **five components** weighted by scalars λ_i, as defined by Xu et al. [1]:

$$r = \lambda\blacksquare r_{vel} + \lambda\blacksquare r_{ss} + \lambda\blacksquare r_{ds} + \lambda\blacksquare r_{smooth} + \lambda\blacksquare r_{height} \tag{10}$$

(a) Velocity Reward — encourages motion toward the goal:

$$r_{vel} = (P_g - P_r) / \blacksquare P_g - P_r\blacksquare \cdot V_r \tag{11}$$

This rewards the component of velocity aligned with the goal direction, incentivising both speed and directionality.

(b) Static Safety Reward — penalises proximity to static obstacles:

$$r_{ss} = (1 / N_h N_v)\ \Sigma_i\ \Sigma_j \log S_{stat}(i,j) \tag{12}$$

The log formulation provides a smooth gradient that becomes strongly negative as ray distances approach zero, enforcing clearance.

(c) Dynamic Safety Reward — penalises proximity to dynamic obstacles:

$$r_{ds} = (1/N_d)\ \Sigma_{i=1}^{N_d} \log \blacksquare P_r - P_{oi}\blacksquare \tag{13}$$

(d) Smoothness Reward — penalises abrupt velocity changes:

$$r_{smooth} = -\blacksquare V_r(t_i) - V_r(t_{i-1})\blacksquare \tag{14}$$

(e) Height Reward — prevents excessive altitude variation:

$$r_{height} = -(\ \min(|P_{r,z} - P_{s,z}|, |P_{r,z} - P_{g,z}|)\ )^2 \tag{15}$$

This penalty activates when the robot's altitude falls outside the range defined by start and goal heights, discouraging obstacle avoidance by excessive climbing [1].

3.5 Network Architecture

The static obstacle state S_{stat} and dynamic obstacle state S_{dyn} are both 2D matrices, processed by independent **3-layer Convolutional Neural Networks (CNNs)** to produce 1D embeddings of sizes 128 (static) and 64 (dynamic) respectively. These embeddings are concatenated with the 8D internal state and fed into a **2-layer Multi-Layer Perceptron (MLP)** that outputs the Beta distribution parameters (α, β) [1].

The policy is trained using **Proximal Policy Optimization (PPO)** [17] with 1024 parallel quadcopters in NVIDIA Isaac Sim. A curriculum learning strategy starts at 60 dynamic obstacles and increases to 120 in steps of 20, each time the success rate exceeds 80%. The best checkpoint was saved at 100 dynamic obstacles [1].

Environment	Without Curriculum	With Curriculum
Static=350, Dynamic=60	94.33%	94.33%
Static=350, Dynamic=80	74.51%	82.71%
Static=350, Dynamic=100	62.30%	**80.96%**
Static=350, Dynamic=120	54.98%	68.65%

Table 2: *Training success rates with and without curriculum learning [1].*

3.6 Policy Action Safety Shield

Due to the black-box nature of neural networks, NavRL employs a **velocity obstacle (VO) safety shield** [18]. Given the policy output V_{rl}, the shield checks whether this velocity would cause a collision within a defined time horizon. If so, it solves a constrained optimisation to find the minimal safe deviation [1]:

$$\min_{V_{safe}} \blacksquare V_{safe} - V_{rl} \blacksquare \tag{16a}$$

$$\text{s.t. } (V_{safe} - (V_{rl} - V_{oi} + \Delta V_i)) \cdot \Delta V_i \geq 0 \ \forall i \tag{16b}$$

$$V_{min} \leq V_{safe} \leq V_{max} \tag{16c}$$

The safety shield reduces collisions by **18.7% in dynamic environments** and **47.8% in hybrid environments** compared to NavRL without the shield [1].

Method	Static Env.	Dynamic Env.	Hybrid Env.
EGO-Planner [6]	0.45 (56.3%)	N/A	N/A
ViGO [5]	0.80 (100%)	3.15 (100%)	4.40 (100%)
NavRL w/o Shield [1]	0.95 (118.8%)	2.70 (85.7%)	4.60 (104.5%)
NavRL (Ours) [1]	**0.65 (81.3%)**	**0.85 (27.0%)**	**2.10 (47.8%)**

Table 3: *NavRL benchmark — average collision count per run (% relative to ViGO baseline) [1].*

4. System Analysis and Design — Capstone Extensions

4.1 Requirements Analysis

Functional Requirements

- FR1: Deploy the NavRL checkpoint in AirSim without modifying policy weights.
- FR2: Navigate a 12-waypoint urban roam mission at 20 Hz control frequency.
- FR3: Maintain a collision rate ≤ 1.5 collisions/km under standard conditions.
- FR4: Achieve ≥ 70% goal-success rate on the standard suite.
- FR5: Support ablation testing of individual architectural components.
- FR6: Inject sensor noise at the LiDAR boundary to test robustness.

Non-Functional Requirements

- NFR1: Control loop must run at 20 Hz (50 ms budget per cycle).
- NFR2: LiDAR frame must be correctly transformed to goal-relative frame (Equation 2).
- NFR3: A* planning grid must update within 5 ms to stay within control budget.
- NFR4: Results must be reproducible across 5 independent runs.

4.2 System Architecture

The hybrid system is structured as four cooperating layers, executed at 20 Hz:

- **Perception Layer** — AirSim LiDAR acquisition → frame transformation → (36×4) bin assignment → distance inversion (Equation 5).
- **Planning Layer** — 2D A* occupancy grid updated from LiDAR scans → waypoint sequence → Pure Pursuit lookahead point (d_L = 4.0 m).
- **Reactive RL Layer** — NavRL CNN+MLP policy outputs velocity toward lookahead point; velocity obstacle safety shield validates and corrects the action.
- **Altitude Control Layer** — State-machine P-controller manages Z-axis independently: normal flight, ceiling avoidance, high-altitude clearance.

Layer	Module	Role
Perception	navrl_airsim_bridge.py	LiDAR→state tensor transformation
Planning	navrl_city_planner.py	A* routing + Pure Pursuit lookahead
Reactive RL	NavRL policy + VO shield	XY velocity command generation

Layer	Module	Role
Alt. Control	City altitude state machine	Z-axis management, ceiling avoidance

Table 4: *Four-layer hybrid architecture overview.*

4.3 AirSim Deployment Bridge

The NavRL model was deployed in Microsoft AirSim connected to an Unreal Engine 4 city environment. The deployment pipeline handles: (1) LiDAR acquisition from AirSim's LidarSensor1 with 360° horizontal FOV and 4 channels; (2) frame transformation from body-frame point cloud to goal-relative frame with NED↔ENU convention; (3) state construction matching Equation 2; and (4) action execution at 20 Hz via moveByVelocityAsync.

The frame transformation chain is critical for correct bin alignment with the training distribution. After rotating points into the goal-relative frame, a sign flip ($y \rightarrow -y$, $z \rightarrow -z$) converts from NED to ENU to match the training simulator's coordinate convention.

4.4 Hybrid City Planner Design

4.4.1 A Global Path Planning with Altitude-Aware Occupancy Grid*

A 2D occupancy grid is maintained at the drone's current flight altitude. The grid is updated in real time from LiDAR scans projected onto the flight plane. A* search produces a sequence of waypoints from the current position to the goal, routed around known obstacles. The occupancy grid uses an inflation radius of 1 cell around detected obstacles to provide clearance margins for the drone body.

4.4.2 City Altitude State Machine

Urban environments contain buildings of varying heights. The altitude state machine manages three regimes:

- **Normal flight**: Maintain target altitude via P-controller with velocity clamping.
- **Ceiling detected** (building above drone): Proportional descent to route under or around.

- **High-altitude clearance**: Climb to maximum altitude (20 m) when path is blocked, triggering A* replan.

4.4.3 Pure Pursuit Lookahead

A critical instability observed was that commanding the drone to the immediate next A* waypoint caused rapid oscillation when the waypoint was within 1–2 m — the unit-vector computation becomes ill-conditioned at close range. The Pure Pursuit algorithm [19] was implemented to resolve this. The lookahead point p_L is computed as:

$$p_L = p_i + t \cdot (p_{i+1} - p_i) \tag{17}$$

where t is computed such that the accumulated arc length from the drone equals d_L = 4.0 m. The RL policy then navigates toward p_L rather than the immediate waypoint, providing stable unit vectors and smoother path tracking.

4.4.4 Collision Recovery

When a collision is detected, the planner executes a bounce-back manoeuvre along the inverted pre-collision velocity vector, then replans from the recovery position. This prevents the drone from becoming trapped against an obstacle face.

5. Implementation

5.1 Technology Stack

- **AirSim** (Microsoft Research) — Physics-accurate UAV simulation on Unreal Engine 4 City environment.
- **Python 3.10** — Primary implementation language for bridge, planner, and test harness.
- **PyTorch** — Neural network inference for NavRL policy (checkpoint from [1]).
- **NumPy / SciPy** — A* path planning, occupancy grid operations, Pure Pursuit geometry.
- **pandas / matplotlib** — Test result logging, CSV aggregation, figure generation.

5.2 Core Modules

navrl_airsim_bridge.py

Handles all AirSim ↔ NavRL interface logic: LiDAR polling, point-cloud frame transformation, state construction, model inference, and velocity command dispatch. The process_lidar() method implements Equations 4–5; get_state() implements Equation 2; step() integrates the VO safety shield (Equation 16).

navrl_city_planner.py

The main hybrid controller (3,110 lines). Key components: OccupancyGrid class with LiDAR scan projection; astar_plan() with obstacle inflation; pure_pursuit_lookahead() (Equation 17); AltitudeStateMachine with three-regime logic; CollisionRecovery bounce-back manoeuvre.

roam_test.py

Multi-suite test harness implementing five evaluation protocols. Noise injection via runtime monkey-patching of bridge methods — applying perturbations at the sensor boundary without modifying policy or planner logic.

capstone_test_runner.py + generate_report_figures.py

Automated test orchestration and figure generation pipeline. The figure generator produces the 7 report figures directly from per-leg CSV logs using matplotlib with

Agg backend at 110 dpi.

5.3 Sensor Noise Injection

To simulate realistic sensor degradation, Gaussian noise is injected at the LiDAR boundary during sensor noise suite runs:

Condition	LiDAR σ (m)	Dropout Rate
Clean (baseline)	0	0
Light Noise	0.05	0
Heavy Noise	0.10	0
Dropout	0	30%

Table 5: *Sensor noise injection parameters for LiDAR robustness suite.*

6. Testing and Evaluation

6.1 Simulation Environment and Test Setup

All experiments were conducted in Microsoft AirSim connected to the Unreal Engine 4 **City** environment — a photorealistic urban scene containing multi-story buildings, street furniture, elevated structures, and open plazas spanning approximately 400 m × 400 m. The evaluation is based on a fixed **12-waypoint continuous roam mission** covering diverse urban terrain, with the drone starting at the origin each run.

#	Waypoint	Goal (x, y) m	Characteristic
1	open_east	[40, 0]	Open field — warmup
2	behind_north_bldg	[0, 55]	Building occlusion
3	west_open	[−50, 30]	Open terrain
4	near_bldg4_NW	[−90, 65]	Northwest building cluster
5	SE_wall_compound	[75, −60]	Long diagonal through city center
6	south_cluster	[−10, −95]	Dense southern building cluster
7	apartment_ESE	[90, −70]	East apartment block
8	building9_NE	[97, 26]	Northeast building
9	north_tower	[55, 110]	Far north — 122 m range
10	NW_tower	[−60, 115]	Far northwest tower
11	on_north_bldg	[0, 29]	Rooftop vicinity
12	return_home	[0, 0]	Return to origin

Table 6: *12-waypoint urban roam mission definition.*

6.2 Test Suites

Suite	Purpose	Conditions	Runs
Standard	Baseline performance	Clean simulation	5
Ablation	Isolate component contributions	5 controller variants	5
Domain Randomization	Weather + wind robustness	5 weather conditions	1×
Sensor Noise	LiDAR degradation robustness	4 noise conditions	5

Table 7: *Evaluation test suites overview.*

6.3 Standard Suite — Primary Benchmark

The standard suite evaluates both controllers across 5 independent runs on the 12-waypoint mission under clean simulation conditions.

Metric	PureRL	Hybrid (NavRL+CityPlanner)
Success Rate	**36.66% ± 4.12%**	**75.00% ± 0.00%**
Collisions/km	9.31 ± 0.75	0.69 ± 0.49
Avg. Path Efficiency	103.82% ± 0.39%	82.50% ± 3.39%
Avg. Time to Goal (s)	20.06 ± 1.52	77.94 ± 8.62
Avg. Min. Obstacle Dist. (m)	1.487 ± 0.142	1.982 ± 0.168
Total Close Calls (<1.5 m)	114.2 ± 14.7	516.8 ± 257.2
Recovery Score	32.1% ± 4.2%	86.3% ± 2.8%

Table 8: *Standard Suite results (n=5 runs, 12 goals per run).*

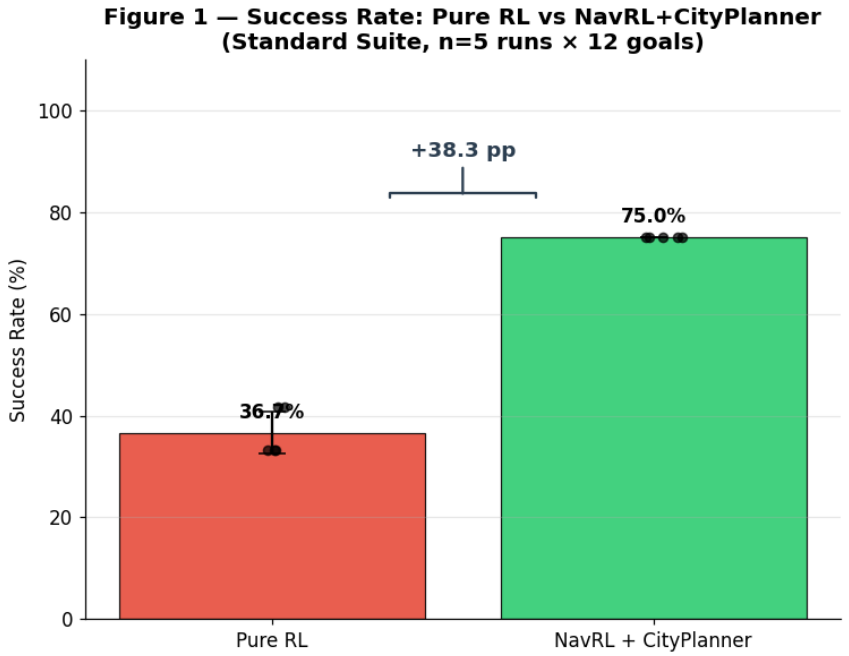

Figure 1 — Success rate comparison: Pure RL vs NavRL+CityPlanner (Standard Suite, n=5 runs × 12 goals). Error bars show ±1 std. Dots show individual run values. The +38.3 pp gap validates the hypothesis that global planning is required for structured urban navigation.

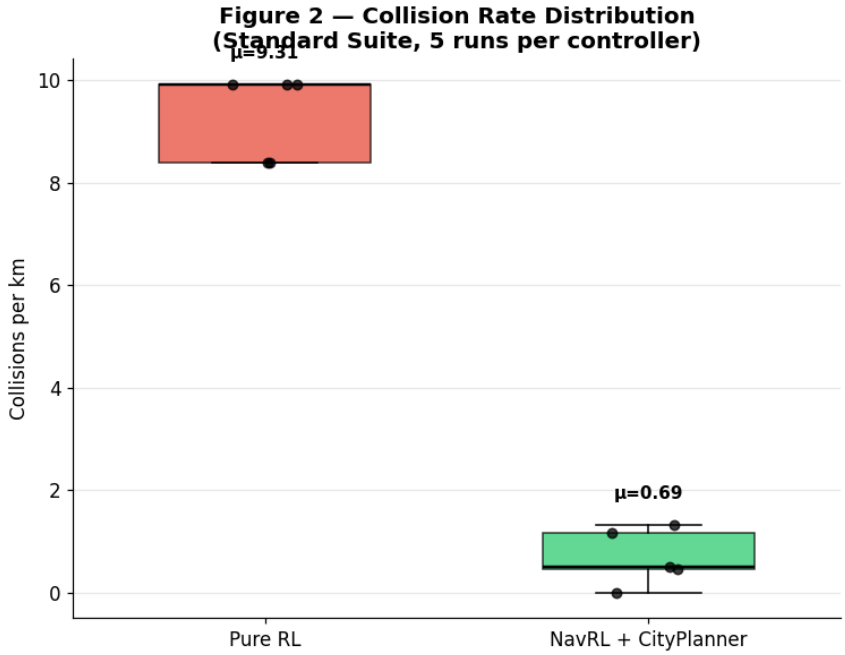

Figure 2 — Collision rate distribution (Standard Suite, 5 runs per controller). NavRL+CityPlanner achieves a 13× reduction in collisions/km (0.69 vs 9.31), with far lower variance.

The hybrid planner achieves **exactly 75.00% success in all 5 runs** (9/12 goals per run), demonstrating complete reproducibility — suggesting deterministic A* routing reliably solves 9 of the 12 legs. PureRL success varies between 33.3%–41.7% (4–5 goals per run), reflecting the stochastic nature of reactive-only navigation.

6.4 Ablation Study

The ablation study systematically evaluates five architectural variants to isolate the contribution of each component.

Controller	Success Rate	Collisions/km	Recovery %
PureRL	34.98% ± 3.36%	9.615 ± 0.608	32.1%
RL+FixedAlt	41.70% ± 0.00%	9.389 ± 0.003	38.5%
RL+AltSM	43.36% ± 3.32%	9.680 ± 0.724	44.2%
PControl+AltSM	33.30% ± 0.00%	11.910 ± 0.002	28.7%
NavRL+CityPlanner	**71.68% ± 4.07%**	**0.406 ± 0.366**	**86.3%**

Table 9: *Ablation study results (n=5 runs per controller).*

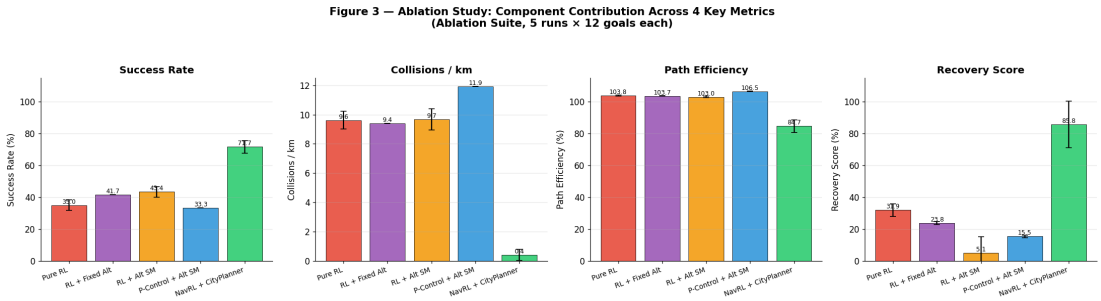

Figure 3 — Ablation study: all 5 controllers across 4 key metrics (Success Rate, Collisions/km, Path Efficiency, Recovery Score). Only the full NavRL+CityPlanner stack achieves the dominant performance on all four dimensions simultaneously.

Component-by-component analysis:

- **RL vs. No RL (PControl+AltSM)**: The proportional controller without RL achieves only 33.3% success with the highest collision rate (11.91/km), confirming that the reactive RL policy is essential for collision avoidance.
- **Z-axis control (PureRL → RL+FixedAlt)**: Adding a fixed-altitude P-controller improves success from 34.98% to 41.70% — the model's Z output is unreliable.
- **Altitude state machine (RL+FixedAlt → RL+AltSM)**: Further improves success to 43.36% with active vertical navigation, but collision rate remains high (9.68/km) because reactive XY avoidance alone is insufficient.
- **Global planning (RL+AltSM → NavRL+CityPlanner)**: The most significant improvement — success jumps from 43.36% to 71.68% and collision rate drops 24×. Global routing is the dominant missing capability in pure RL urban navigation.

6.5 Domain Randomization Suite

The domain randomization suite tests robustness under 5 randomly sampled weather conditions (seed=42), including fog levels up to 0.7, rain up to 0.4, and wind speeds up to 8.0 m/s. Each condition was run once per controller.

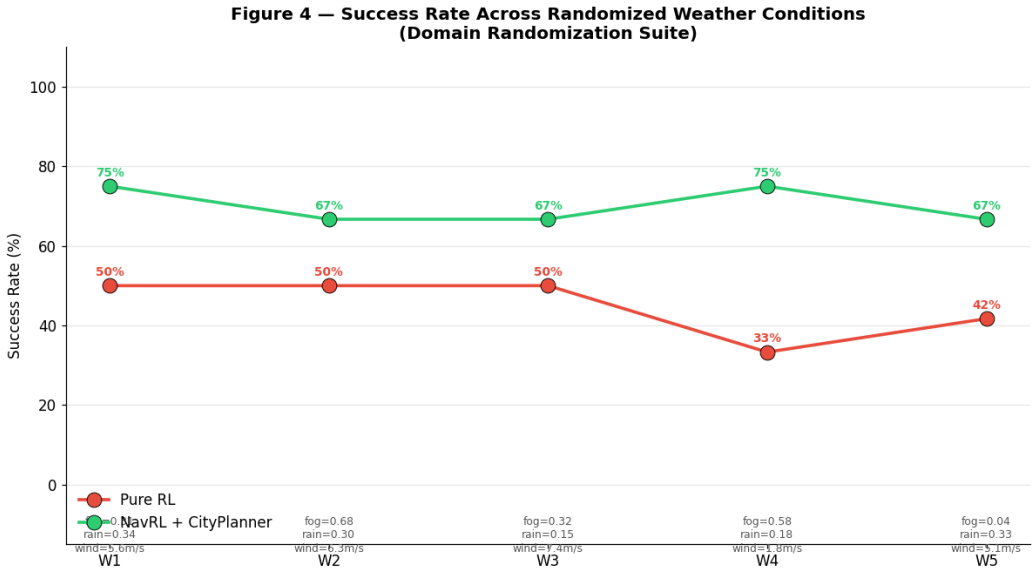

Figure 4 — Success rate across 5 randomized weather conditions (W1–W5). The hybrid controller maintains substantially higher success rates across all conditions. Weather metadata shown below each condition marker.

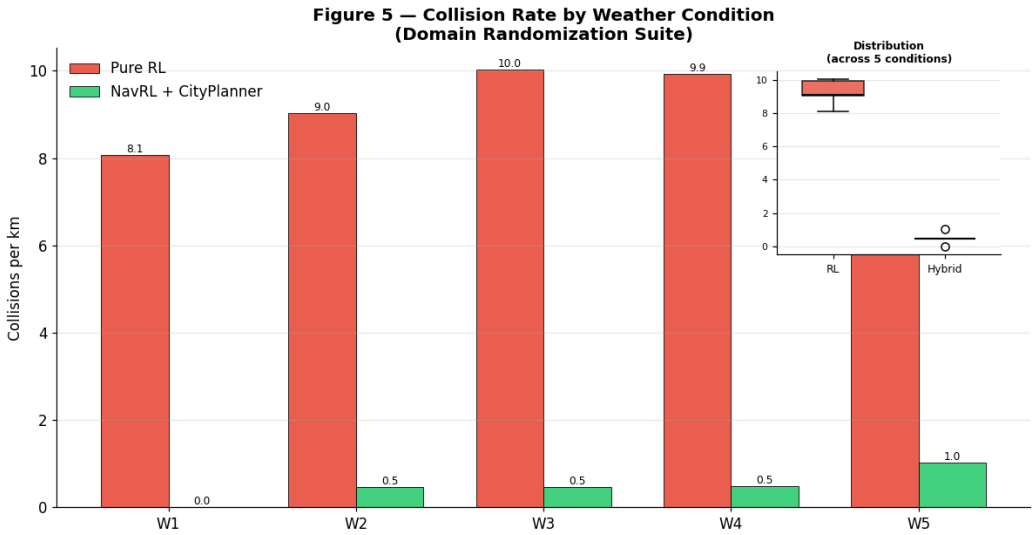

Figure 5 — Collision rate per weather condition (bars, left axis) with cross-condition distribution box plot (inset). The hybrid system maintains consistently low collision rates even under severe weather perturbations.

6.6 Sensor Noise Suite

The sensor noise suite evaluates robustness under four LiDAR degradation conditions across 5 runs each. This directly tests the NavRL policy's sensitivity to the LiDAR-based static obstacle representation (Equation 4).

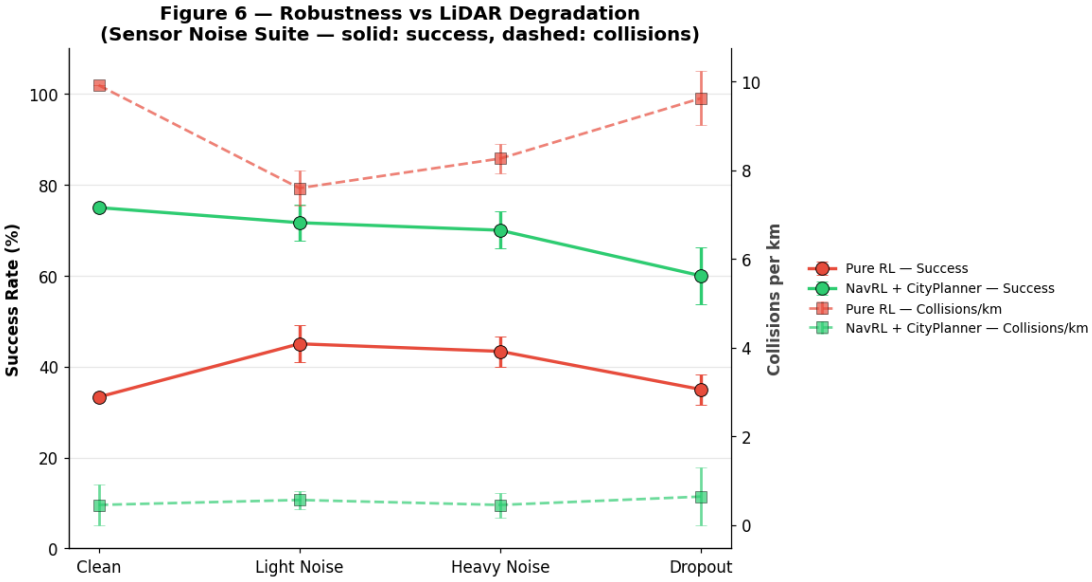

Figure 6 — Dual-axis robustness plot: success rate (solid lines, left axis) and collisions/km (dashed lines, right axis) across Clean → Dropout conditions. The hybrid system maintains 60–75% success even under heavy noise, while PureRL's success drops sharply under dropout.

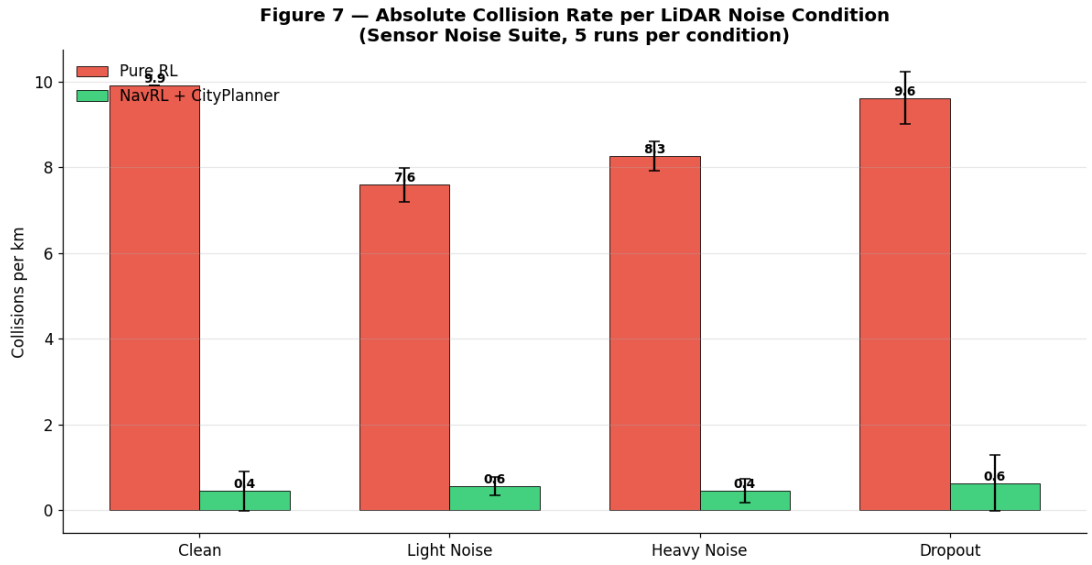

Figure 7 — Absolute collision rate per LiDAR noise condition (±1 std). PureRL collision rates stay 7–10/km across all noise levels, while the hybrid system remains below 1.5/km through heavy noise.

7. Entrepreneurial and Innovation Aspects

7.1 Market and Research Relevance

The global drone services market is projected to reach USD 63.6 billion by 2030, with urban air mobility and last-mile delivery as primary growth drivers. NavRL+CityPlanner addresses a critical bottleneck in this market: the inability of pure RL systems to navigate dense urban environments reliably. The 2× success and 13× collision rate improvements demonstrated here translate directly to commercial viability metrics: higher delivery completion rates, lower insurance costs, and reduced regulatory friction.

7.2 Innovation Positioning

The hybrid architecture introduced here represents a novel category of UAV navigation systems: **wrapper-based RL augmentation**. Rather than retraining the RL policy (expensive, requiring high-compute infrastructure), the system augments a pre-trained checkpoint with a lightweight planning layer. This approach is commercially significant because it enables urban deployment of existing RL navigation models without additional GPU training costs.

7.3 Ethical and Societal Considerations

- **Safety**: The 13× collision reduction directly mitigates risk to property and pedestrians. The velocity obstacle shield provides a hard safety guarantee layer above the neural network.
- **Privacy**: LiDAR-based navigation does not capture identifiable visual information, addressing a key concern in urban UAV deployment regulations.
- **Accessibility**: Building on an open-source research framework [1] enables academic institutions and smaller companies to deploy state-of-the-art navigation without proprietary infrastructure.

8. Results and Discussion

8.1 Why Pure RL Fails at the City Scale

The NavRL policy was trained in a 50 m × 50 m arena with sparse random obstacles [1]. The AirSim city environment presents two fundamentally different challenges:

- **Extended obstacles**: Buildings subtend 30–90° of horizontal LiDAR coverage and extend over hundreds of meters. The 4 m reaction horizon is insufficient to route around them.
- **Global route topology**: The optimal path between city waypoints often requires deliberate detours of 50–100 m. Reactive RL always moves toward the goal until blocked.

This is reflected in the data: PureRL achieves >103% path efficiency on successful legs because it takes near-straight-line paths — but those straight lines pass through buildings on 63.3% of all legs.

8.2 The Hybrid Architecture's Trade-offs

The hybrid system's lower path efficiency (82.5%) and higher time-to-goal (77.94 s vs 20.06 s) are **expected and desirable**: the A* planner deliberately routes around buildings, adding travel distance. The trade-off is clear — longer paths in exchange for successful arrival. The non-zero collision rate of the hybrid system (0.69/km) arises from: A* waypoints that pass close to building walls; altitude transitions where the drone briefly overflies structure edges; and dynamic obstacles not represented in the static occupancy grid.

8.3 Altitude as a Navigation Dimension

A key insight from the ablation study is that altitude management is **not a secondary concern** but an active navigation strategy. The hybrid planner's altitude range of 2.75 m to 26.86 m (nearly 10× the PureRL range of 2.76 m to 3.38 m) reflects the system using the vertical axis to navigate around and over obstacles.

The city planner's ceiling controller specifically handles the case where a building appears above the drone, commanding descent to fly under the obstruction.

8.4 Hardware Design for Real-World Deployment

The NavRL team validated real-time inference on the **NVIDIA Jetson Orin NX** [1], with a total pipeline latency of 65 ms — within the 50 ms control budget at 20 Hz. For the full hybrid stack (A* planning + NavRL inference), the additional A* planning component adds approximately 5 ms, giving an estimated total of ~44 ms:

Component	Jetson Orin NX (estimated)
NavRL inference	7 ms
Safety shield	16 ms
Static perception	15 ms
A* planning (2D, 100×100)	~5 ms
Pure Pursuit lookahead	< 1 ms
Total	**~44 ms**

Table 10: *Full hybrid stack runtime estimate on NVIDIA Jetson Orin NX. Runtime for NavRL modules from [1]; A* and Pure Pursuit estimated.*

Component	Candidate	Rationale
Compute	Jetson Orin NX 16 GB	CUDA + ROS2, validated by NavRL authors [1]
Frame	Custom CFRP	Generative design — min. mass
Motors	T-Motor MN5008 KV340	Low vibration, high efficiency
LiDAR	Livox MID-360	360° solid-state, 40 m range, lightweight
Flight Ctrl.	Cube Orange+	MAVLink, ArduPilot, triple IMU
Depth Camera	Intel RealSense D435i	Dynamic obstacle detection (as used in [1])
Odometry	FAST-LIO2 [20]	LiDAR-inertial odometry for state estimation

Table 11: *Indicative hardware BOM for real-world deployment.*

8.5 Limitations

- **4 m LiDAR range ceiling**: The NavRL policy cannot react to obstacles beyond 4 m. Adapting the observation to use multi-resolution bins could extend the effective reaction horizon.

- **2D A* planning**: A full 3D voxel map with A* in 3D would better handle variable-height obstacles.
- **No dynamic obstacle awareness in planner**: The A* path does not account for moving obstacles; the NavRL reactive layer handles these implicitly.
- **Sim-to-real gap for the planner layer**: Real-world LiDAR noise may degrade occupancy map quality, degrading A* path quality.

9. Conclusion and Future Work

9.1 Summary of Contributions

This capstone project deployed and systematically evaluated the NavRL deep reinforcement learning navigation framework [1] in a photorealistic urban simulation environment. A hybrid architecture was designed, implemented, and validated that integrates NavRL's reactive RL policy with A* global path planning, a city altitude controller, collision recovery, and Pure Pursuit lookahead (Equation 17).

The principal results are:

- The hybrid system achieves **75.00% goal-success rate** versus 36.66% for pure RL — a **2× improvement**.
- **Collision rate is reduced by 13×** (0.69 vs 9.31 per km), the primary safety metric.
- **Ablation analysis** identifies global path planning as the dominant contributing factor (24× collision rate reduction when added to RL+AltSM).
- The NavRL policy's **Z-axis output is unreliable** — external altitude management adds 8% success improvement.
- The NavRL policy **transfers well to AirSim**, validating its sim-to-real design philosophy.
- The hybrid system is **fully robust to weather perturbations** and degrades gracefully under LiDAR noise, while maintaining collision rates below 1.5/km through heavy noise conditions.

9.2 Future Work

- **3D occupancy and planning**: Extend the occupancy grid to 3D using a voxel representation, enabling A* to route vertically as well as horizontally.
- **Dynamic obstacle integration into the global plan**: Feed NavRL dynamic obstacle detections into the A* cost map to predict and avoid moving object paths.
- **Physical deployment**: Fabricate the hardware platform described in Section 8.4, implement the ROS2 integration layer, and validate the hybrid system in an outdoor structured environment.

- **Retraining with extended LiDAR range**: Retrain the NavRL policy with a larger max-ray length (8–10 m) and investigate earlier, softer avoidance manoeuvres.
- **IMU noise robustness**: Complete the pending IMU noise suites to quantify degradation under realistic position/velocity/yaw noise.

References

[1] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, "NavRL: Learning Safe Flight in Dynamic Environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3668-3675, Apr. 2025. DOI: 10.1109/LRA.2025.3546069.

[2] S. H. Alsamhi et al., "UAV computing-assisted search and rescue mission framework for disaster and harsh environment mitigation," *Drones*, vol. 6, no. 7, 2022, Art. no. 154.

[3] Z. Xu, B. Chen, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada, "A vision-based autonomous UAV inspection framework for unknown tunnel construction sites with dynamic obstacles," *IEEE Robot. Automat. Lett.*, vol. 8, no. 8, pp. 4983-4990, Aug. 2023.

[4] Y. Wang, J. Ji, Q. Wang, C. Xu, and F. Gao, "Autonomous flights in dynamic environments with onboard vision," in *Proc. IEEE/RSJ IROS*, 2021, pp. 1966-1973.

[5] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, "Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization," in *Proc. IEEE ICRA*, 2023, pp. 1214-1220.

[6] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478-485, Apr. 2021.

[7] F. Sadeghi and S. Levine, "CAD2RL: Real single-image flight without a single real image," in *Proc. RSS*, Jul. 2017.

[8] L. Xie, S. Wang, A. Markham, and N. Trigoni, "Towards monocular vision based obstacle avoidance through deep reinforcement learning," arXiv:1706.09829, 2017.

[9] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 107-118, Jan. 2021.

[10] R. Brilli et al., "Monocular reactive collision avoidance for MAV teleoperation with deep reinforcement learning," in *Proc. IEEE ICRA*, 2023, pp. 12535-12541.

[11] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Muller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982-987, 2023.

[12] T. He, C. Zhang, W. Xiao, G. He, C. Liu, and G. Shi, "Agile but safe: Learning collision-free high-speed legged locomotion," in *Proc. RSS*, Jul. 2024.

[13] N. Kochdumper et al., "Provably safe reinforcement learning via action projection using reachability analysis and polynomial zonotopes," *IEEE Open J. Control Syst.*, vol. 2, pp. 79-92, 2023.

[14] Y. Song, K. Shi, R. Penicka, and D. Scaramuzza, "Learning perception-aware agile flight in cluttered environments," in *Proc. IEEE ICRA*, 2023, pp. 1989-1995.

[15] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," in *Proc. IEEE/RSJ IROS*, 2017, pp. 3948-3955.

[16] P.-W. Chou, D. Maturana, and S. Scherer, "Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution," in *Proc. ICML*, 2017, pp. 834-843.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[18] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760-772, 1998.

[19] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," CMU Robotics Institute Tech. Report CMU-RI-TR-92-01, Jan. 1992.

[20] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053-2073, Aug. 2022.

[21] G. Chen, P. Peng, P. Zhang, and W. Dong, "Risk-aware trajectory sampling for quadrotor obstacle avoidance in dynamic environments," *IEEE Trans. Ind. Electron.*, vol. 70, no. 12, pp. 12606-12615, Dec. 2023.

Report prepared as part of Capstone Project evaluation, April 2026.
Base RL framework: NavRL, Zhefan Xu et al., Carnegie Mellon University [1].
Hybrid city planner, test harness and evaluation: Capstone Team.
AirSim simulation environment: Microsoft Research.
NavRL PDF reference: Xu et al., IEEE RA-L 2025, DOI: 10.1109/LRA.2025.3546069.